

COMPILER DESIGN PROJECT

PHASE-2

Glyph: A Compiler for a Multi-Byte Unicode
Token Language

By: Sriranjani K (24BCE0946)

Objective

To transition Glyph from a lexical design concept into a fully functioning computational engine. This phase covers the implementation of the core compiler modules: Lexical Analysis, Syntax Analysis, Abstract Syntax Tree (AST) construction, Semantic Scope Management, and a Runtime Execution Engine (Interpreter).

Expanded Language Specification

Symbol	Codepoint	Token	Meaning
	U+1F5A8	PRINT	Output statement
	U+2795	PLUS	Addition
	U+2796	MINUS	Subtraction
	U+2716	MUL	Multiplication
	U+2797	DIV	Division
	U+1F7F0	EQ	Equality comparison (IF / LOOP conditions)
	U+1F501	LOOP	Loop construct
	U+2753	IF	Conditional construct
	U+1F4E5	ASSIGN	Assigns a value to a variable
	U+1F431	VAR (cat)	Variable identifier
	U+1F436	VAR (dog)	Variable identifier
	U+1F438	VAR (frog)	Variable identifier
	U+1F3ED	DEF	Begins a function definition
 END	U+1F51A	END	Closes a block (DEF, IF, LOOP)
 BACK	U+1F519	RETURN	Returns a value from a function
	U+1F98A	FUNC (fox)	Function-name identifier
	U+1F981	FUNC (lion)	Function-name identifier
	U+1F43B	FUNC (bear)	Function-name identifier

Core Implementation Modules

To ensure high code quality and modularity, the C implementation was strictly divided into decoupled subsystems.

1. Lexical Analyzer (glyph_lexer.c)

Unlike traditional compilers that consume single-byte ASCII streams, Glyph operates on multi-byte UTF-8 sequences. The lexer reads the entire source file into a buffer and extracts Unicode codepoints mathematically using bitwise shifts.

It dynamically classifies these codepoints into Tokens, separating Variable Identifiers (🐱) from Function Identifiers (🐱) to lay the groundwork for scope resolution.

Key Implementation Snippet:

```
static uint32_t decode_utf8(const unsigned char *s, int *len) {
    unsigned char c = s[0];
    if (c < 0x80) { *len = 1; return c; }
    else if ((c & 0xE0) == 0xC0) { *len = 2; return ((c & 0x1F) << 6) | (s[1] &
0x3F); }
    /* ... bitwise logic for 3 and 4 byte graphemes ... */
}
```

2. Syntax Analyzer & AST (glyph_parser.c & glyph_ast.c)

A custom recursive-descent parser written to enforce grammar rules. Rather than executing code immediately, the parser evaluates token precedence and constructs an **Abstract Syntax Tree (AST)**. Control flow constructs (IF, LOOP, DEF) generate NODE_BLOCK structures, recursively nesting statements via a linked list until a TOK_END terminator is matched.

Key Implementation Snippet: AST Node Structure

```
typedef struct Node {
    NodeType type;
    long num_value;
    int id;
    struct Node *left;
    struct Node *right;
    struct Node *body;
    struct Node *next;
} Node;
```

3. Semantic Analyzer & Symbol Table

Scope and Symbol Table management is handled dynamically. The compiler enforces strict namespace and scoping rules:

- Functions exist in a **Global Symbol Table**.
- Variables exist in isolated **Local Scopes**. A variable used in one function shares no memory with a variable of the same emoji in another function.
- Semantic checks ensure variables cannot be read in an expression before they are formally assigned (TOK_ASSIGN).

4. Runtime Execution Engine / Interpreter (*glyph interpreter.c*)

Fulfilling the syllabus requirement for a runtime execution environment, we implements a Tree-Walking Interpreter that evaluates the AST using a Call Stack. When a function is called, the engine pushes a new **Activation Record** onto the stack, initializes a clean local scope, passes arguments to the parameters, and

executes the function body.

Upon hitting `NODE_RETURN`, it pops the Activation Record and passes the result back to the caller.

Key Implementation Snippet: Call Stack & Activation Records

```
#define MAX_CALL_DEPTH 100
```

```
typedef struct {  
    long vars[3];    /* Holds values for local variables */  
    int assigned[3]; /* Semantic tracking to prevent unassigned reads */  
} ActivationRecord;
```

```
static ActivationRecord stack[MAX_CALL_DEPTH];  
static int sp = -1; /* Current Stack Pointer */
```

Functional Demonstration & Test Cases

The following test cases demonstrate the successful integration of the Lexer, Parser, and Execution Engine.

Test Case 1: Arithmetic & Function Call (Scope Isolation)

This test validates that functions can be defined, accept parameters, mutate local variables, and return values to the global scope.

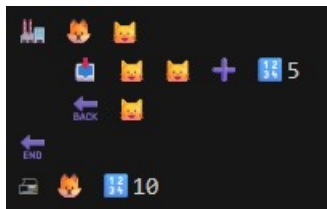


Figure 1: *input.glf*

```
long fox(long cat) {
    cat = cat + 5;
    return cat;
}

printf("%ld\n", fox(10));
```

Figure 2: Equivalent C Code

Output:

```
--- Compiling and Executing: input.glf---
15
```

Result: PASSED. The interpreter successfully allocated an Activation Record, passed 10 to the local variable cat, added 5, and printed the returned result.

Test Case 2: Conditionals and Equality

This test validates the IF parser logic, the EQ (=) operator, and the block-terminator END (←_{END}).



Figure 3: input.glf

```
long lion(long dog) {  
    if (dog == 100) {  
        return 1;  
    }  
    return 0;  
}  
printf("%ld\n", lion(100));
```

Figure 4: Equivalent C Code

Output:

```
--- Compiling and Executing: input.glf---  
1
```

Result: PASSED. The AST evaluated the nested NODE_EQ condition, correctly routed the control flow into the IF block, and returned early.

Intermediate Results (AST Representation)

To prove the parser correctly builds intermediate structures rather than just evaluating strings, the internal Abstract Syntax Tree generation was validated. For the assignment statement    5 +  3 (where cat = 5 + 3), the parser correctly groups the binary operation below the assignment node:

```
[NODE_ASSIGN: var_id=0]
└─ [NODE_BINOP: op='+']
    ├── [NODE_NUM: val=5]
    └─ [NODE_NUM: val=3]
```

Conclusion and Future Roadmap

What's been done so far

To date, the Glyph project has successfully evolved from a conceptual language design into a fully functional, Turing-complete computational engine. The primary technical hurdle which is lexing multi-byte UTF-8 grapheme clusters in C without relying on standard ASCII-based tools was successfully solved.

Now, the compiler pipeline was fully modularized and expanded to include a recursive-descent parser and an Abstract Syntax Tree (AST) generator. Furthermore, to elevate the project to a publishable standard and implement successfully measures will be taken as needed.

Future Work

With the core execution engine complete, we will focus heavily on system validation, error handling, and final documentation as required by the project.

Potential Enhancements

While the current implementation is robust, here are some technical enhancements that could be explored to further optimize the language:

1. Zero-Width Joiner (ZWJ) Support:

Currently, the lexer handles multi-byte single codepoints perfectly. Future iterations could add logic to group ZWJ sequences (e.g., parsing 🧑🏽👉🏻🧑🏿 as a single valid token rather than separate emojis).

2. Target Code Emission:

Expanding the backend pipeline to translate the optimized AST/TAC directly into runnable C or Python source code, providing an alternative to the built-in runtime interpreter.

3. Data Type Expansion:

Extending the semantic analyzer and runtime memory to support strings, booleans, and arrays alongside the current integer implementations.